

Why a robot needs a separate controller

M6 • 25 Feb 2027 - 22 Mar 2027 • 75 hours

Outcome of the month

Create an ESP32-S3 controller that polls an IMU and encoder, transmits consistent measurement snapshots, limits the control signal and enters a predefined, tested state on a fault. This forms the basis of the electronics for a leg or finger: the computer may delay a message, while the local controller must determine whether measurements are fresh and operation may continue. Deliver firmware, a wiring diagram, bus traces, cycle-timing measurements and a record of injected faults. Completing the module requires demonstrated controller operation on a physical test rig.

What to bring from previous months

Use the M1 telemetry format and error handling, circuit and measurement fundamentals from the electrical modules, and the M5 leg CAD model for sensor placement and cable routing. Entry check: explain voltage relative to common ground, an input versus a power output, the purpose of a pull-up and encoder-angle units. Find the power circuit and pins reserved for USB, memory and boot configuration in your exact board's documentation. If this is still difficult, include the review in the first 2 hours, before connecting external circuits.

The minimum physical setup depends on available components: ESP32-S3, one compatible IMU, an angle source or encoder, a button, a protected low-power output indicator and measuring equipment. Check exact models first; select components according to their specifications. Temporarily replace a missing sensor with a packet generator that keeps the same interface. This demonstrates program logic, but the hardware item remains open in the final report until measured on a real device.

Scope and connection to robots

M6 does not require moving a loaded leg or assembling the entire robot. Test the enable output on a low-power load; the motor rig and its limits belong to M7. The same mechanisms will detect missing finger feedback in a hand, or interrupted angle and IMU updates on a walking platform. Software inhibition of a command and independent removal of energy are separate checks. A stopping circuit must account for the fact that removing power from a real vertical axis can cause it to fall; here, test its electrical logic on the rig.

Preparing verification evidence for G1 is part of this work. The G1 integration review itself has a separate 35 hours in the overall plan and is excluded from these 75 hours. Take all hardware limits from the selected components' documentation and the load calculation; the exercise frequencies below define a measurement scenario, rather than a safe operating regime for the future robot.

Organising 75 hours and the test rig

Four work blocks

Block A, hours 1-20: 10 hours of GPIO, PWM, ADC, timer and memory theory; 10 hours of wiring, signal measurement and interrupt practice. Block B, 21-40: 10 hours of protocols and 10 hours of IMU, encoder and robust communication work. Block C, 41-60: 10 hours of task scheduling and 10 hours of cycle-time variation (jitter) measurement, queue checks and watchdog testing. Block D, 61-75: 5 hours of fault analysis and 10 hours of integration tests. This gives exactly 35 hours of study and analysis plus 40 hours of practice.

Split each 10 hours of theory into five 2-hour sessions: 35 minutes reading the relevant section, 35 minutes drawing or calculating, 40 minutes on a small verification example and 10 minutes of notes. Schedule ten-hour practical blocks on Sundays; divide them into stages with breaks and count the 10 hours as active work. Split the final 5 analysis hours into 2+2+1. Calendar boundaries may overlap adjacent modules; retain rest days, holidays and G1 from the overall plan. Complete numbered steps during available study hours without counting any hours twice.

Deliverable structure

Create firmware/ for ESP-IDF, host/ for the Python decoder, config/ for pin assignments and limits, tests/ for frames and state transitions, evidence/ for traces, and plots/ for graphs. In README, state board model, revision, SDK version, build and flashing commands, external power wiring and each sensor type. Keep raw measurements separate from processed plots, so errors can be traced to the device or script. For every run, record run_id, firmware hash, configuration, start time and workload type.

Change one thing at a time

In each hardware experiment, change one parameter: bus frequency, polling period, queue depth or logging load. Before starting, record the expected effect and termination condition. Afterwards, record actual frame count, errors and missed deadlines. If a trace is unclear, return to the last reproducible configuration. Do not change wiring, firmware and communication speed simultaneously and then claim to have identified the cause. Use separate commits for the working baseline and deliberate fault injection.

Take the first wiring photograph with ground, power, data lines and enable output labelled. In the log, distinguish “physically measured”, “verified by a software test” and “awaiting hardware verification”. This allows work to continue while components are delayed, followed by the remaining hardware checks.

GPIO, PWM and ADC: signals in a real robot

What to study

Study GPIO modes: input, output, open drain, pull-up/pull-down and active level. For PWM, relate frequency, period and duty cycle: $\text{duty} = \text{high-level time} / \text{period}$. For ADC, distinguish raw code, pin voltage and the physical quantity measured by the sensor. Resolution alone does not guarantee accuracy: know the permissible input range, calibrate the channel and assess noise. Read the board pin table and GPIO restrictions in the ESP-IDF GPIO reference; check PWM parameters in the ESP-IDF LEDC reference. A motor or coil is not an ordinary GPIO load.

Task 1. Button-controlled motion enable

The aim is to verify operator enable for a local joint controller. Initial conditions: a low-power indicator replaces the actuator, and the active-level circuit is explicitly known. 1. Create an input/state/output table. 2. Configure a safe electrical state before application startup. 3. Implement time-based debouncing while retaining raw events for analysis. 4. Perform 30 presses and releases, including short presses and holds. Save the diagram, log and state-machine test. Check for one confirmed action per press and no accidental enable at startup. Fault: a disconnected button wire must not be interpreted as a new start command.

Task 2. Check the control pulse

The aim is to check whether the software command matches the physical signal. Use only a protected test output. 1. Select frequency and resolution within peripheral limits. 2. Apply several duty-cycle values within the API's permitted range. 3. Capture at least 100 periods per mode with an analyser or oscilloscope. 4. Calculate mean, minimum and maximum period and duty deviation. Save the trace and a table of commands and measured values. Separately test zero command and an invalid parameter; the driver must report failure rather than silently retaining the previous active output.

Task 3. Analogue diagnostic channel

This task prepares for supply-voltage or current measurement, initially using a known permissible test signal. Record the channel transfer function and units. Collect a series with constant input, calculate mean and standard deviation, then change the signal to at least three points across the range and compare with independent measurements. Save conversion coefficients, calibration residuals and a saturation flag. For a signal outside the measurable range, the program marks data invalid rather than substituting a plausible value. Never apply an ADC voltage outside your exact board's limits.

Experiment references: [GPIO](#), [PWM](#). Use the configuration and API error sections for your pinned ESP-IDF version.

Timers, memory and interrupt handlers

Essential theory for reliable firmware

Distinguish a hardware timer, the system tick and elapsed-time measurement. Schedule periodic polling from its expected time, otherwise execution duration is added to every period. Learn which operations are permitted in an ISR, how to pass an event to a task, and why logging or waiting for a lock in a handler increases latency. Check the available ESP-IDF APIs in the documentation for the chosen version. A pointer to a local array cannot be used after that array's lifetime ends; volatile provides neither atomicity nor synchronisation between cores and tasks.

Task 4. Timestamp an encoder event

The aim is to associate joint angle with measurement time, rather than USB packet reception time. Begin with a pulse generator or available encoder. 1. Define a sequence with a known edge count. 2. In the ISR, save only the minimum: a count or timestamp and a task notification. 3. Process and transmit data outside the ISR. 4. Compare received events with the original trace. Deliver an interval plot, lost-event count and an explanation of counter overflow. Increase frequency until an error is detected, within the electrical rig's capabilities; the resulting limit applies to this configuration.

Task 5. Who owns the measurement snapshot?

The aim is to prevent memory overwrites from making angle and IMU readings refer to different times. Create a Snapshot structure with seq, sample time, measurement fields and validity bits. 1. First reproduce the error in a computer test: the producer changes a shared buffer and the consumer reads it later. 2. Pass the snapshot by value through a queue, or use buffers with explicit ownership. 3. Populate fields with related check values and verify that slowing the consumer does not produce mixed snapshots. Save the integrity test and data-lifetime description. Define queue-overflow behaviour in advance and add a loss counter.

Stack, heap and limited resources

Fix queue sizes and estimate memory per snapshot. Measure the minimum remaining task stack with the SDK's available tools after the highest-load test, rather than just after startup. Check memory-allocation failure and driver reinitialisation paths. Do not add dynamic allocation to a frequent loop without a reason; begin with bounded structures. State units and clock type for every timeout. Test timer-counter overflow using artificial values in a computer test rather than waiting for it on the board. The report must distinguish the largest measured latency from a proven real-time guarantee.

Experiment references: [timer](#), [encoder](#). Use the configuration and API error sections for your pinned ESP-IDF version.

Buses and packets: IMU, encoder and computer

Interfaces to study

UART: baud rate, character format, message boundaries, complete and partial frames. I2C: address, register operations, ACK/NACK, pull-ups and timeout. SPI: CS, CPOL/CPHA, bit order and register reads. Study CAN and RS-485 in terms of transceiver role, topology and termination; do not confuse CAN arbitration with shared-channel management in RS-485. Implementing all five physical buses within the month is unnecessary: primary computer communication and one real sensor interface are required; reinforce the others by interpreting traces and drawing a small circuit.

Task 6. Identify the sensor from its trace

The aim is to confirm that data comes from the intended IMU and is decoded correctly. Inputs: the installed sensor's datasheet, identification register and measurement format. 1. Check power and logic levels. 2. Read the identifier and range settings. 3. Capture a transaction with the analyser and manually label address, direction, data and acknowledgement. 4. Convert bytes to a number, accounting for signedness and byte order. Save an annotated trace and a calculation converting the raw value to physical units. Fault: an incorrect address or broken line must produce a timeout and missing-data status, rather than an endless wait.

Task 7. A telemetry frame that can resynchronise

Design a bounded-length frame: start marker or boundary encoding, version, length, seq, timestamp, payload and checksum. Choose a CRC algorithm from a documented implementation and state its parameters; a simple sum has different error-detection properties. 1. Write encoding and decoding code on the computer. 2. Feed the stream one byte at a time and in random chunks. 3. Add a missing byte, junk before a frame, incorrect length and an altered checksum. 4. Connect the same decoder to the board. Acceptance: no damaged frame in the fixed test set is accepted, and the next intact frame is recovered without restarting.

Throughput and units

Estimate traffic before flashing firmware: frame bytes $\times$ frames per second plus the chosen transport's overhead. Compare the calculation with the actual trace and retain capacity for diagnostics. For each field, state SI unit, sign, range and invalid flag; do not mix rad/s and deg/s. For the encoder, separately state counts per revolution, accounting for decoding mode and gearbox. An open computer connection does not make the last value fresh: assess measurement freshness from sample time and sequence number.

Experiment references: [UART](#), [I2C](#), [SPI](#). Use the configuration and API error sections for your pinned ESP-IDF version.

FreeRTOS: measuring timing behaviour

Separate responsibilities

Divide the work into four functions: sensor reading, state and enable checks, telemetry transmission and service diagnostics. Assign priorities according to the consequences of delay, rather than code size. First achieve predictable execution with a small number of tasks; introduce core pinning only after measurement. Read the differences in ESP-IDF FreeRTOS and its critical-section rules in ESP-IDF FreeRTOS: Critical Sections. A queue transfers data, a mutex protects a shared resource, and a notification signals an event; choose the mechanism to suit communication and synchronisation requirements.

Task 8. Measure period, latency and deadline

Choose an exercise period T , for example 10 ms for the first low-power rig, then justify it using sensor frequency and permitted communication time. The planned start of cycle k is $t_0 + kT$. Record actual start s_k and finish e_k : start jitter = $s_k - (t_0 + kT)$, execution time = $e_k - s_k$. A deadline is missed if completion occurs after the selected deadline. 1. Capture a baseline run. 2. Repeat with intensive logging and a slowed receiver. 3. Compare p50, p95, p99, maximum and missed-deadline count over equal numbers of cycles. Save raw timestamps, configuration and plots. An average must not hide a rare, large outlier.

Task 9. Queue overflow without losing control

The aim is to keep updating joint state while USB communication is slow. Create a fixed-length queue and two modes: full data recording for a rig experiment, or retaining only the latest state for monitoring. Each mode needs its own explicitly described loss policy. Slow the consumer enough to guarantee the queue fills. Check maximum occupancy, dropped_count, seq ordering and time to detect stale data. Firmware must not wait indefinitely for queue space in a task responsible for inhibiting the output. Save the observed limit together with test duration.

Task 10. Test the watchdog

Study the interrupt watchdog and task watchdog in ESP-IDF Watchdogs. On the low-power rig, deliberately block a selected task in a separate test build and record the reset cause. Measure the enable output using an independent channel before reset and after boot. Acceptance: restarting must not restore active enable from a saved command; a new explicit start procedure is required after successful checks. Watchdog servicing must reflect the health of critical functions, rather than run unconditionally from an unrelated timer. Separately stop the computer process: this is a communication failure, not necessarily a failure of the MCU itself.

Integrated rig and fault state machine

controller-io v1 specification

Rig inputs: IMU, encoder angle or count, operator command and electrical-limit sensors where available. Outputs: telemetry snapshot, diagnostic events and test-channel enable. Initial mode BOOT validates configuration and moves to DISARMED. Only an explicit start command with all conditions satisfied enters ARMED. Loss of valid measurements, violation of a monitored limit or E-stop enters FAULT. Removing the cause does not by itself enter ARMED. Recovery proceeds through a confirmed fault reset and DISARMED.

Task 11. Transition table and its code checks

Inputs are the lists of events, states and physical outputs. 1. For every state/event pair, specify the next state, reason code and channel enable. 2. Implement a pure transition function that can be tested on the computer. 3. Issue commands out of order: start during a fault, repeated reset, connection restored without confirmation. 4. Compare actual telemetry with the table. Save the table, tests and logs for at least five event sequences. Check that no path leads from BOOT or FAULT to ARMED without the required conditions and a new operator action.

Task 12. Independent stop

Draw the energy path from source to test load separately from the software command path. Identify a component capable of stopping the energy supply independently of the normal MCU loop; select it from the load and circuit documentation. First test its function on a low-power equivalent. Then hold the program in a deliberate test hang and activate the physical stop. Deliver the circuit, output measurement and recovery procedure. Measure time from the physical event to cessation of the test signal and compare it with the rig requirement set in advance. This test does not assign an industrial safety rating to a homemade system.

Configuration and reset during supply loss

Store configuration with a schema version and range validation. Do not save ARMED as a state that restores automatically. For NVS, study initialisation, writing and errors in ESP-IDF NVS. Test a corrupt or unknown format on a data copy: loading invalid limits must inhibit enable. Test power reset by controllably disconnecting an appropriate source with a low-power load, rather than by short-circuiting it. After power returns, the log shows the restart cause, configuration is checked again and motion enable remains absent. Actual power-stage tolerance of voltage dips is tested later with the specific M7 actuator.

Test matrix and verification criteria

Input-data failures

Test F1: disconnect the IMU. Record the last valid snapshot time, timeout and removal of enable. Choose the freshness threshold from the expected update period and the rig's acceptable delay. Test F2: corrupt frame and false length. The decoder rejects the packet without reading beyond the buffer, increments a counter and recovers the next valid frame. Test F3: a stuck encoder value while packet sequence numbers continue. Demonstrate the difference between fresh data and a physically plausible measurement; without a motion model, a constant value alone does not prove a fault.

Execution and recovery failures

Test F4: the telemetry receiver stops reading. Logging drops data according to the selected policy while freshness checks and channel shutdown continue; measure response latency. Test F5: the monitored task is blocked. The watchdog triggers in the test build, the reset cause is included in the report, and the controller remains DISARMED after boot. Test F6: E-stop while the MCU is hung. The independent circuit switches off the test output; releasing the button does not restart it. For F5/F6, capture the physical output, because "FAULT" in the console does not prove the signal has stopped.

Configuration and timing boundaries

Test F7: configuration contains a negative period, unknown version or nonnumeric coefficient. Each cause has clear diagnostics; reject the entire configuration or apply it using predefined atomic-update rules. Test F8: power restart while ARMED with a low-power load. A new enable is required after startup. Test F9: artificial counter overflow and two frames with the same seq. Statistical counters must not report negative age, and duplicates must not conceal missing new measurements. Test F10: ADC measurement reaches the range boundary; the program reports saturation rather than inferring an exact current value.

How to record each result

For every scenario, record circuit and firmware versions, initial state, fault-injection method, expected transition, measured times, filenames and outcome: pass, fail or "not physically verified". Repeat critical transitions several times and retain their variability; one successful reset does not characterise reliability. For each timing metric, state its start, end and measurement clock. Do not subtract PC and MCU timestamps without synchronisation or an offset estimate. If an instrument is unavailable, the computer test remains useful, but leave the physical-latency item open.

Work sequence: hours 1-40

Hours 1-10: read and design

1-2: describe rig components and parameters; check pin assignments, levels and unused pins. Draw the power path and initial output state. 3-4: study GPIO and PWM signals, calculate period and duty cycle, and select measurable modes. 5-6: cover ADC and unit conversion; tabulate raw and converted values. 7-8: study timers and ISR; draw an event-timestamp-task-packet timing diagram. 9-10: understand snapshot memory and queues; write a shared-buffer overwrite scenario. Finish the block with an interface design and a list of expected measurements.

Hours 11-20: first practical block

11-12: assemble and test the button with protected indicator; save photographs and the task 1 state table. 13-14: perform task 2 PWM measurements, including invalid configuration. 15-16: collect task 3 analogue series and plot calibration. 17-18: measure task 4 edges and move processing out of the ISR. 19-20: correct snapshot ownership in task 5, add a bounded queue and save the working baseline. Allocate the final 15 minutes of every two-hour step to documenting results; those minutes are included in the stated time.

Hours 21-30: data and protocols

21-22: study UART and frame boundaries; estimate required throughput. 23-24: study I2C and the specific IMU's registers; manually decode an example transaction. 25-26: study SPI and compare clocking modes with the sensor datasheet; do not connect a new sensor before checking levels. 27-28: create a comparative diagram of UART, I2C, SPI, CAN and RS-485: lines, addressing, shared-medium access, transceiver and termination. 29-30: design framing, CRC and resynchronisation rules; prepare a fixed set of damaged messages and expected responses.

Hours 31-40: the complete sensor path

31-32: read IMU identification and measurements; verify axes and sign. 33-34: capture and annotate task 6 traces; compare raw values and SI conversions. 35-36: implement task 7 frame reception with partial reads. 37-38: inject corruption, disconnect and reconnect the sensor; remove indefinite waits. 39-40: perform an extended run in the chosen mode and compare frame count and traffic volume with calculations. Save the second baseline as baseline-2 and describe its limitations. If new hardware is unavailable, use a test-data generator and label these steps as software work, rather than a completed physical sensor path.

Work sequence: hours 41-75

Hours 41-50: timing, resources and faults

41-42: read the FreeRTOS task model and ESP-IDF differences; draw dependencies and selected priorities. 43-44: allocate available time within a cycle and define task 8 metrics; choose frequency from the sensor and available communication. 45-46: compare queues, notifications and mutexes for your data; examine slow logging. 47-48: study watchdogs and reset causes; define testable boot behaviour. 49-50: study configuration storage and separate recovery after power loss into distinct states. Do not spend these hours optimising code before obtaining measurements.

Hours 51-60: measurement practical

51-52: implement periodic scheduling and timestamp recording. 53-54: perform the task 8 baseline run and repeat under load; produce distribution plots. 55-56: overflow the queue in task 9 and compare the selected policies. 57-58: inject the task 10 hang in a separate build and observe the physical output during reset. 59-60: check restart, memory and configuration; fix one identified cause of deadline misses and repeat only the affected experiment. Finish the block by stating measured limits, test durations and what remains unproven.

Hours 61-65: five hours of analysis

61-62: analyse rig hazards. For each, identify the energy source, failure event, possible consequence, preventive measure and an observable check of that measure's operation. 63-64: complete the task 11 state-machine table and task 12 independent-path diagram; verify that hardware and software stops do not depend on the same hung task. 65: prepare the acceptance sequence, initial states and record forms. This 2+2+1 split retains the overall 35/40 balance and adds no hours to the programme.

Hours 66-75: final rig block

66-67: integrate the state machine, measurement-freshness checks and configuration validation. 68-69: perform F1-F3 and F7; compare messages with the transition table. 70-71: perform F4-F5 and check latency under load. 72-73: independently verify F6/F8 on a low-power load and test recovery; check completeness of timestamps and raw traces. 74-75: reproduce the project from a clean build, make a short demonstration video and assemble the G1 package. Explicitly state any hardware check not completed. You may omit a second transport prototype, an extended PC interface and extra plots; the fault design, primary sensor path and prevention of automatic restart are compulsory.

Acceptance, self-check and handover to M7

Completion conditions

First: firmware builds from the instructions and starts without active output enable. Second: one real IMU and angle-measurement source have documented formats, units and timestamps; temporary substitutes are explicitly marked. Third: physical traces of the primary communication path and damaged-frame checks are available. Fourth: cycle period, jitter, execution time and missed deadlines have been measured under a described workload. Fifth: watchdog, lost communication and E-stop do not cause automatic restart after recovery. Sixth: the independent stopping path has been verified from the physical output, and the limits configuration cites the source of every value.

Set numerical requirements for frequency, timeouts and time to remove the test signal before the experiment, based on the rig's purpose. Place requirement, measurement, instrument, conditions and conclusion side by side in the final table. Where evidence is missing, "not completed" is more useful than an average without its source. Before G1, prepare the revision-labelled circuit, photographs, repository, tests, raw measurements, plots, failed-case log and list of unverified hardware assumptions.

Questions for oral review

1. Why does volatile not resolve a race between ISR and task? 2. How does measurement age differ from packet age? 3. Where should a timeout occur if the IMU does not respond? 4. Why does CRC not establish the physical plausibility of an angle? 5. What happens to the output while the bootloader runs? 6. Why must watchdog servicing depend on critical tasks? 7. How can p99 look good despite a dangerous single outlier? 8. What exactly does the physical stop switch off during a program hang? 9. Which data may be restored from NVS? 10. Why does power restoration not mean permission to resume motion?

Answer each question using your own circuit, test or trace. If your answer uses only library terminology, create a physical example: a missing finger-angle measurement, a stale leg command or USB overloaded by logging. Connecting a cause to its observable effect is this month's central skill.

What the next module needs

Pass the working controller, measurement and command interfaces, timeout list, reset log, fault state machine and independent-stop circuit to M7. Adding a real actuator requires new current, thermal and mechanical limits; low-power rig values do not transfer automatically. The completed interface will support actuator-latency measurement and reproducible identification experiments. Save release controller-io-v1 and the list of required hardware checks separately from subsequent control changes.

Assigned reading: a focused reading route

Read only the specified sections before the corresponding experiment. Most literature is in English; keep explanations, the glossary and the report in English for this edition. Reading and preparation are included in the existing 75 hours; you do not need to work through entire books.

Core reading and limits on transfer

1. Jonathan W. Valvano, **Embedded Systems: Introduction to ARM Cortex-M**

Microcontrollers. Chapters 4, 6, 9-11: input/output, pointers and data structures, interrupts and real time, analogue interfaces, communication. For tasks 1-7, read the principles; do not apply Cortex-M register descriptions to ESP32-S3 or buy a TI development board for the book.

2. Richard Barry / FreeRTOS, Mastering the FreeRTOS Real Time Kernel. Current online book: chapters 4, 5, 7: Task Management, Queue Management, Interrupt Management. Tasks 8-9 need task states, queues and deferred ISR event processing. Older PDFs use different numbering; follow these chapter titles.

3. Edward A. Lee, Sanjit A. Seshia, Introduction to Embedded Systems, 2nd ed. (2017). Chapter 3 Discrete Dynamics; chapter 11 Multitasking. Read selectively on finite-state machines and concurrent tasks. For task 10, draw the fault-machine transitions; for the queue, explain the race and data ownership. Full formal verification of the controller is outside this module.

Documentation instead of API guesses

ESP-IDF FreeRTOS. Tasks, SMP Scheduler, Critical Sections. After the book, compare the specific ESP-IDF implementation: stack-size units, cores and locks. Select the reference version matching the project's pinned version.

Failure and recovery. Watchdogs: IWDG, TWDT, JTAG and NVS: Read, Write and Erase; Error Handling. For task 10, check hangs, restarts and corrupted configuration; ARMED must not restore automatically.

Buses and peripherals. Links for GPIO, LEDC, timer, UART, I2C, SPI and PCNT appear on the relevant practical pages. Add your IMU's datasheet: registers, units, byte order and access conditions.

Reading deliverables. Prepare a pin diagram, a task-period table, a buffer-ownership diagram and a fault state machine. Every document should explain an actual trace or reproducible failure.

Glossary: terms and definitions

Microcontroller (MCU). A chip containing processor, memory and peripherals for direct device control; ESP32-S3 uses Xtensa cores.

GPIO. A programmable general-purpose digital pin; the controller and board circuit determine its mode, permissible voltage and functions.

Interrupt. An event that invokes a handler outside normal task execution; service latency depends on priorities and blocking.

ISR. An interrupt service routine with bounded execution time and restricted permitted operations; it usually delegates lengthy processing to a task.

Timer. A hardware or software mechanism for measuring time; a hardware counter and software handler have different sources of delay.

Jitter. Variation in start times or operation duration relative to a selected reference; measured in units of time, such as microseconds.

Deadline. The time by which a computation or action must finish; a miss is defined by system requirements, rather than average performance.

RTOS task. An independent execution context with a stack and state; the scheduler selects a ready task according to priorities.

Queue. A bounded buffer for passing messages between contexts; overflow and waiting require an explicitly selected policy.

Data race. Unsynchronised concurrent access to shared memory involving a write; the result may depend on execution order.

Critical section. Code accessing shared state protected by a coordinated synchronisation mechanism; its duration affects system latency.

DMA. Transfer between peripherals and memory without per-item processor copying; buffer ownership and transfer completion must be coordinated.

UART. An asynchronous serial interface with agreed baud rate and character format; the protocol specifies message boundaries separately.

I2C. An addressed synchronous bus with SDA data and SCL clock lines; its electrical implementation requires checks of pull-ups, capacitance and levels.

SPI. A synchronous serial interface with clock and device selection; mode, bit order and transaction boundaries must match the sensor.

CRC. A check code for detecting specified transmission errors; it provides neither sender authenticity nor protection against intentional substitution.

Data age. Time from acquisition of a physical measurement to its use, including transmission, queues and processing.

Watchdog. A mechanism detecting the absence of expected evidence of operation; a controller restart does not guarantee a safe power-stage state.

Finite-state machine. A model with a finite set of states and defined event-driven transitions, useful for enabling operation and handling faults.

NVS. Nonvolatile value storage with an access API; configuration needs a version and validity checks after loading.

Equipment, installation and readiness checks

Complete preparation within the first study blocks in place of a repeated general overview. The total remains 75 hours; waiting for delivery is excluded. The statuses below identify actions for you to take, rather than purchases or installations already completed.

USE EXISTING EQUIPMENT AND BUY BEFORE THE PRACTICAL

Reuse the Mac M4, multimeter, supply, breadboard, wiring and M04 low-power load. **Buy: 2 boards** [ESP32-S3-DevKitC-1](#) of one verified revision: the first is the controller, the second a pulse generator and packet-exchange device. Check Flash, PSRAM, circuit and pins reserved for USB and boot configuration.

1 IMU breakout module: access to raw three-axis accelerometer and gyroscope data, documented registers, I2C and/or SPI, an accessible interrupt line, and supply and logic levels compatible with 3.3 V. Check driver support for ESP32-S3, the chosen interface and ESP-IDF version. If you already have such a sensor, do not buy another. A sensor providing only processed angles does not replace access to raw channels.

1 logic analyser: at least 8 digital channels, inputs suitable for the chosen levels, UART/I2C/SPI decoders and a sample rate with margin for the bus being studied. An example is [Saleae Logic 8](#); check Apple Silicon software before ordering. **1 USB set:** two data cables with connectors matching the boards, an adapter or hub if needed, probes with common ground and connectors.

INSTALL NOW - macOS ARM

Start with [PlatformIO IDE for VS Code](#): build, upload and monitor a simple project. Then use [ESP-IDF, official Get Started](#) for target esp32s3 with compatible CMake, Ninja and compiler tools. Pin versions; the PlatformIO espressif32 platform version and ESP-IDF framework version are different fields and their compatibility requires a separate check.

Add [KiCad for macOS](#) for the wiring diagram and electrical checks, and software for your exact analyser, for example [Logic 2 for Saleae](#). A sigrok-compatible instrument may need [PulseView](#); check ARM and USB support before purchase. Logic 2 does not support arbitrary analysers. Use Ubuntu 24.04 ARM64 in UTM only after separately checking USB; the main build and measurement route is native Mac.

INSTALLATION CHECK

On both boards, perform build→flash→monitor, hardware reset and firmware-version reading. Then test UART loopback or communication between boards, measure PWM, perform one I2C or SPI transaction with the IMU and export the trace. Verify raw-count units and monotonic time. Open the circuit in KiCad and run ERC; save environment versions and the pin-assignment diagram.

Optional, for later: STM32 only for a separate comparison; the motor with encoder is purchased in M07. In M06 an existing encoder can be the angle source, and the second board can generate quadrature pulses. The generator tests the interface; required real sensor measurements remain separate acceptance items.

Motion fundamentals: angle, velocity and time

Required: connect measurements with motion

The leg controller receives counts, but motion is described by angle $q(t)$ and its change over time. Before the experiment, explain: $90^\circ = \pi/2$ rad; positive rotation is defined by the oriented joint axis, and an identical angle in two packets does not prove a stop. In M6, use the available sensor or a generator on the second ESP32-S3, without a new motor.

Measurement model and worked example

Let c_k be an unwrapped counter without overflow, C the counts per revolution of the measured shaft after the selected decoding, and t_k the sample time in seconds from one MCU clock. Relative angle $q_k = 2\pi(c_k - c_0)/C$, in rad. If the encoder is on the motor, converting to link angle separately requires a known gear ratio and sign. There is no reduction here.

Exercise data: $C=4000$, $c=(100;104;110)$, $t=(0;0.010;0.025)$ s. This gives $q=(0;0.006283;0.015708)$ rad. Interval velocity $v_k = (q_k - q_{k-1})/(t_k - t_{k-1})$ is 0.628319 rad/s in both intervals. Dividing the second increment by the nominal 0.010 s gives a false 0.942478 rad/s: interval variation creates the appearance of acceleration.

Finite-difference velocity is the interval average, rather than the exact instantaneous derivative. One count corresponds to 0.001571 rad; over short windows, quantisation is visible in velocity. Smoothing reduces noise but introduces delay. USB reception time is no substitute for physical sampling time: a late, intact packet may contain a stale angle.

One end-to-end experiment for tasks 4, 5 and 8

First reproduce the table on the computer. Then transmit a known A/B sequence from the second board and compare received edges with the logic analyser. Save counts, sample_time, receive_time, seq, q and interval v. Plot velocity calculated from actual and nominal intervals. Adding a constant to every c must not change relative angles.

Repeat with delayed consumer reads and identical timestamps on two frames. Reject a zero or negative interval before dividing; a stale snapshot does not become fresh because of a new packet. For example, at constant $v=0.628319$ rad/s, age 0.05 s means an angle change of 0.031416 rad. This indicates the error scale, rather than specifying a robot timeout.

Included in the 75 hours: 1 hour from 7-8 for the time model; 1 hour from 17-18 for task 4; 1 hour from 53-54 for task 8 plots. Use their existing traces and report. The generator validates the signal path; the real angle sensor requires a separate check.

Read selectively: [Lee/Seshia, Introduction to Embedded Systems, ch. 3 Discrete Dynamics](#) for state and samples; [ESP-IDF PCNT: Introduction, Channel Actions](#) for A/B counting semantics. Check the API against the pinned version.